

LambdaScript: A Functional Programming Language

Alex Kozik (ajk333@cornell.edu)

Department of Computer Science, Cornell University, Ithaca, NY, USA

Abstract

[Write a 150-word abstract summarizing: (1) What LambdaScript is, (2) Key features (type inference, pattern matching, recursive types), (3) Main contributions of the work]

Keywords: functional programming; type inference; pattern matching; Hindley-Milner; algebraic data types

1 Introduction

2 Language Design and Formal Semantics

2.1 Syntax

We present the abstract syntax of LambdaScript using BNF notation. The language supports first-class functions, algebraic data types, pattern matching, and polymorphic type inference.

Expressions:

$$\begin{aligned}
 e \in \mathbf{Exp} \quad e ::= & n \mid b \mid s \mid c \mid () \\
 & \mid x \mid \lambda p:\tau.e \mid e_1 e_2 \\
 & \mid \text{let } p = e_1 \text{ in } e_2 \\
 & \mid \text{let rec } p = e_1 \text{ in } e_2 \\
 & \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \\
 & \mid \text{case } e \text{ do } \{p_i \rightarrow e_i\}_{i=1}^n \\
 & \mid [] \mid e_1 :: e_2 \mid [e_1, \dots, e_n] \\
 & \mid (e_1, \dots, e_n) \\
 & \mid e_1 \oplus e_2
 \end{aligned}$$

where $n \in \mathbb{Z}$ (integers), $b \in \{\mathbf{true}, \mathbf{false}\}$ (booleans), s ranges over strings, c ranges over characters, x ranges over identifiers, and $\oplus \in \{+, -, *, /, \%, ==, !=, <, >, \leq, \geq, \&\&, ||\}$.

Patterns:

$$\begin{aligned}
 p \in \mathbf{Pat} \quad p ::= & n \mid b \mid s \mid c \mid () \\
 & \mid x \mid _ \mid [] \mid p_1 :: p_2 \\
 & \mid (p_1, \dots, p_n) \\
 & \mid C p \mid C
 \end{aligned}$$

where C ranges over data constructors.

Types:

$$\begin{aligned}
 \tau \in \mathbf{Type} \quad \tau ::= & \mathbf{int} \mid \mathbf{bool} \mid \mathbf{string} \mid \mathbf{char} \mid \mathbf{unit} \\
 & \mid \alpha \mid \tau_1 \rightarrow \tau_2 \mid [\tau] \\
 & \mid (\tau_1, \dots, \tau_n) \\
 & \mid T\langle \tau_1, \dots, \tau_n \rangle \\
 & \mid \mu T.\tau \\
 \sigma ::= & \tau \mid \forall \alpha.\sigma
 \end{aligned}$$

where α ranges over type variables, T ranges over type constructors, $\mu T.\tau$ represents recursive types, and σ represents polymorphic type schemes.

2.2 Type System

LambdaScript uses a Hindley-Milner style type system with support for polymorphism and algebraic data types. We define typing judgments of the form $\Delta; \Gamma \vdash e : \tau$, where Δ is a type variable environment, Γ is a term variable environment mapping identifiers to type schemes, and τ is a monomorphic type.

Typing Rules:

Literals:

$$\frac{}{\Delta; \Gamma \vdash n : \mathbf{int}} \quad \frac{}{\Delta; \Gamma \vdash b : \mathbf{bool}} \quad \frac{}{\Delta; \Gamma \vdash s : \mathbf{string}} \quad \frac{}{\Delta; \Gamma \vdash () : \mathbf{unit}}$$

Variables:

$$\frac{x : \sigma \in \Gamma \quad \tau = \text{inst}(\sigma)}{\Delta; \Gamma \vdash x : \tau}$$

where $\text{inst}(\forall \alpha.\sigma) = \sigma\{\tau'/\alpha\}$ for fresh τ' .

Functions:

$$\frac{\Delta; \Gamma, x : \tau_1 \vdash e : \tau_2}{\Delta; \Gamma \vdash \lambda x:\tau_1.e : \tau_1 \rightarrow \tau_2}$$

Application:

$$\frac{\Delta; \Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Delta; \Gamma \vdash e_2 : \tau_1}{\Delta; \Gamma \vdash e_1 e_2 : \tau_2}$$

Let Binding:

$$\frac{\Delta; \Gamma \vdash e_1 : \tau_1 \quad \Delta; \Gamma, x : \text{gen}(\Delta, \Gamma, \tau_1) \vdash e_2 : \tau_2}{\Delta; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2}$$

where $\text{gen}(\Delta, \Gamma, \tau) = \forall\{\alpha_i\}.\tau$ and $\{\alpha_i\}$ are type variables in τ but not free in Γ .

Conditional:

$$\frac{\Delta; \Gamma \vdash e_1 : \text{bool} \quad \Delta; \Gamma \vdash e_2 : \tau \quad \Delta; \Gamma \vdash e_3 : \tau}{\Delta; \Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}$$

Pattern Matching:

$$\frac{\Delta; \Gamma \vdash e : \tau \quad \forall i. p_i : \tau \rightsquigarrow \Gamma_i \quad \Delta; \Gamma, \Gamma_i \vdash e_i : \tau'}{\Delta; \Gamma \vdash \text{case } e \text{ do } \{p_i \rightarrow e_i\}_{i=1}^n : \tau'}$$

Lists:

$$\frac{}{\Delta; \Gamma \vdash [] : [\alpha]} \quad \frac{\Delta; \Gamma \vdash e_1 : \tau \quad \Delta; \Gamma \vdash e_2 : [\tau]}{\Delta; \Gamma \vdash e_1 :: e_2 : [\tau]}$$

2.3 Static Semantics

Type inference in LambdaScript follows a constraint-based approach in the style of Algorithm W. The type checker operates in three phases:

1. Constraint Generation

For each expression e , we generate a type τ_e and a set of constraints C . The function $\text{generate}(\Gamma, e)$ returns a pair (τ, C) where:

- τ is the inferred type (possibly containing fresh type variables)
- $C = \{(\tau_i, \tau'_i)\}$ is a set of type equations to be satisfied

Example constraint generation rules:

$$\begin{aligned} \text{generate}(\Gamma, n) &= (\text{int}, \emptyset) \\ \text{generate}(\Gamma, x) &= (\text{inst}(\Gamma(x)), \emptyset) \\ \text{generate}(\Gamma, e_1 e_2) &= (\alpha, C_1 \cup C_2 \cup \{(\tau_1, \tau_2 \rightarrow \alpha)\}) \end{aligned}$$

where $(\tau_1, C_1) = \text{generate}(\Gamma, e_1)$, $(\tau_2, C_2) = \text{generate}(\Gamma, e_2)$, and α is fresh.

2. Unification

The unification algorithm solves constraint sets by computing a most general unifier (MGU). Given constraints $C = \{(\tau_i, \tau'_i)\}$, we compute a substitution S such that $S(\tau_i) = S(\tau'_i)$ for all i .

The unification rules are standard:

- $\text{unify}(\alpha, \tau) = [\tau/\alpha]$ if $\alpha \notin FV(\tau)$

- $\text{unify}(\tau_1 \rightarrow \tau_2, \tau'_1 \rightarrow \tau'_2) = S_2 \circ S_1$ where $S_1 = \text{unify}(\tau_1, \tau'_1)$ and $S_2 = \text{unify}(S_1(\tau_2), S_1(\tau'_2))$
- $\text{unify}(T\langle\tau_1, \dots, \tau_n\rangle, T\langle\tau'_1, \dots, \tau'_n\rangle) = S_n \circ \dots \circ S_1$ where S_i unifies $S_{i-1}(\tau_i)$ with $S_{i-1}(\tau'_i)$

3. Generalization

After solving constraints, we generalize types by introducing universal quantifiers for type variables not appearing free in the environment:

$$\text{gen}(\Gamma, \tau) = \forall\{\alpha_i\}.\tau$$

where $\{\alpha_i\} = FV(\tau) \setminus FV(\Gamma)$.

2.4 Dynamic Semantics

We define the operational semantics of LambdaScript using big-step (natural) semantics. Evaluation judgments have the form $\rho \vdash e \Downarrow v$, where ρ is an environment mapping variables to values, e is an expression, and v is a value.

Values:

$$v ::= n \mid b \mid s \mid c \mid () \mid \langle \rho, \lambda p.e \rangle \mid [] \mid v_1 :: v_2 \mid (v_1, \dots, v_n) \mid C \ v \mid C$$

Evaluation Rules:

Literals and Variables:

$$\frac{}{\rho \vdash n \Downarrow n} \quad \frac{}{\rho \vdash b \Downarrow b} \quad \frac{x \mapsto v \in \rho}{\rho \vdash x \Downarrow v}$$

Functions:

$$\frac{}{\rho \vdash \lambda p.e \Downarrow \langle \rho, \lambda p.e \rangle}$$

Application:

$$\frac{\rho \vdash e_1 \Downarrow \langle \rho', \lambda p.e \rangle \quad \rho \vdash e_2 \Downarrow v_2 \quad \text{match}(p, v_2) = \rho'' \quad \rho' \cup \rho'' \vdash e \Downarrow v}{\rho \vdash e_1 e_2 \Downarrow v}$$

Let Binding:

$$\frac{\rho \vdash e_1 \Downarrow v_1 \quad \text{match}(p, v_1) = \rho' \quad \rho \cup \rho' \vdash e_2 \Downarrow v_2}{\rho \vdash \text{let } p = e_1 \text{ in } e_2 \Downarrow v_2}$$

Conditional:

$$\frac{\rho \vdash e_1 \Downarrow \text{true} \quad \rho \vdash e_2 \Downarrow v}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v} \quad \frac{\rho \vdash e_1 \Downarrow \text{false} \quad \rho \vdash e_3 \Downarrow v}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v}$$

Pattern Matching:

$$\frac{\rho \vdash e \Downarrow v \quad \text{match}(p_i, v) = \rho' \quad \rho \cup \rho' \vdash e_i \Downarrow v'}{\rho \vdash \text{case } e \text{ do } \{p_j \rightarrow e_j\}_{j=1}^n \Downarrow v'}$$

where i is the first index such that $\text{match}(p_i, v)$ succeeds.

Binary Operators:

$$\frac{\rho \vdash e_1 \Downarrow n_1 \quad \rho \vdash e_2 \Downarrow n_2 \quad n_3 = n_1 \oplus n_2}{\rho \vdash e_1 \oplus e_2 \Downarrow n_3}$$

Pattern Matching Function:

The auxiliary function $\text{match}(p, v)$ returns an environment binding variables in pattern p to corresponding values in v , or fails if the pattern does not match:

$$\text{match}(x, v) = \{x \mapsto v\}$$

$$\text{match}(_, v) = \emptyset$$

$$\text{match}(n, n) = \emptyset$$

$$\text{match}(p_1 :: p_2, v_1 :: v_2) = \text{match}(p_1, v_1) \cup \text{match}(p_2, v_2)$$

$$\text{match}((p_1, \dots, p_n), (v_1, \dots, v_n)) = \bigcup_i \text{match}(p_i, v_i)$$

3 Implementation

3.1 Architecture Overview

3.2 Lexer and Parser

3.3 Type Inference Engine

3.4 Pattern Matching

3.5 Evaluator

3.6 Advanced Features

4 Examples and Applications

5 Discussion

5.1 Known Limitations

5.2 Related Work

5.3 Future Work

6 Conclusion